Consultas do gateway LoRa

Fluxo

O cliente Python envia o comando por **UDP ao IP do gateway, porta 4210**. O gateway encaminha a consulta pelo LoRa ao receptor indicado, recebe os fragmentos da resposta e devolve o resultado ao **IP e porta do mesmo cliente**.

```
status:RX01
Python -> UDP -> TX01 -> LoRa -> RX01 -> LoRa -> TX01 -> UDP -> Python
```

Não existe mais o envio periódico de `Ola Mundo`, broadcast de demonstração ou resposta espontânea dos receptores. O tráfego LoRa de requisição, resposta, confirmação e repetição acontece somente por causa de uma consulta remota.

Sintaxe

Comando enviado ao gateway	Quem responde
`status`	O próprio gateway.
`status:TX01`	O próprio gateway, se TX01 for seu endereço atual.
`status:RX01`	RX01, por LoRa.
`temp:RX02`	Sensor interno da CPU de RX02.
`ram:RX01`	Memória de RX01.
`battery:RX01`	Leitura VBAT de RX01.
`status_json:RX02`	Diagnóstico de RX02 em JSON.
`name:RX01`	Endereço configurado em RX01.

Aceita maiúsculas/minúsculas e espaços nas extremidades. Um comando por datagrama. O destino precisa ser um endereço individual; `TODOS` não é aceito. Cada receptor deve ter endereço único. Sem sufixo, os dados são do gateway. O Python sempre usa o IP do gateway, não o IP Wi-Fi dos receptores.

Catálogo comum do gateway e dos receptores

Acrescente `:RX01` ou outro endereço a qualquer consulta para executá-la no receptor.

Comando	Resposta
`help`	Lista iniciada por `COMANDOS`, um comando por linha.
`name`	Endereço local configurado.
`alive`	`alive`.

Comando	Resposta
`info` / `status`	Resumo do dispositivo consultado.
`status_json` / `info_json`	Resumo em JSON.
`build` / `version`	Nome do sketch e data/hora de compilação do dispositivo consultado.
`reason`	Motivo do último reset.
`uptime`	Tempo ligado em segundos.
`temp`	Temperatura interna da CPU em °C.
`cpu`	Modelo, revisão, núcleos, frequência e temperatura.
`ram`	Heap total, livre, usado, mínimo histórico, maior bloco e PSRAM.
`heap`	Heap livre em KB, no estilo do M5NanoC6.
`flash`	Capacidade física, frequência, firmware, partição atual e tamanho do slot OTA.
`net_info`	SSID, IP, gateway, máscara, DNS, MAC, RSSI e dados do AP local.
`ip` / `mac` / `rssi`	Informações da rede do dispositivo consultado.
`radio` / `radio_status`	Modo do driver, código, fase, tentativa, parâmetros e última recepção.
`counters`	Transações completas/falhas, repetições, CRC, timeouts, comandos, respostas e fragmentos.
`battery` / `vbat`	Tensão estimada, ADC em mV, validade, idade e calibração da leitura.
`ota_status`	Suporte, estado e progresso do serviço OTA local.
`udp_status`	Gateway: listener e contadores UDP; receptor: informa que usa LoRa via gateway.

Aliases: `ajuda` e `?` para help; `lora` para radio; `batt` e `bateria` para battery. As consultas não reiniciam a placa, não alteram saídas e não apagam preferências.

Disponibilidade real: o receptor reconhece o mesmo catálogo, mas não passa a ter servidor OTA ou listener UDP. `ota_status:RX01` informa que OTA não está implementado; `udp_status:RX01` informa o transporte LoRa. `flash:RX01` ainda mostra a capacidade física e a partição de aplicação existentes na placa.

Cliente Python

Substitua o IP de exemplo pelo IP do gateway na página 3 do OLED:

```
python tools/udp_query.py 192.168.1.50 name:RX01
python tools/udp_query.py 192.168.1.50 status:RX01
python tools/udp_query.py 192.168.1.50 temp:RX01 ram:RX01 flash:RX02
python tools/udp_query.py 192.168.1.50 battery:RX01 status_json:RX02
python tools/udp_query.py 192.168.1.50 status
```

O cliente usa a biblioteca padrão do Python. O timeout padrão é **75 segundos para consultas com destino** e **3 segundos para consultas locais**; altere com `--timeout`. O parser também formata JSON em comandos como `status_json:RX01`. Cada consulta usa um socket próprio para evitar confundir respostas atrasadas com as de outro comando. Respostas iniciadas por `ERRO:` fazem o cliente retornar código de saída não zero.

Fragmentos e limites

- Entrada UDP: até 255 bytes ASCII; comando remoto: até 128 bytes.
- Resposta: até 1400 bytes, reconstituída antes de ser devolvida em um datagrama UDP.
- Payload LoRa por fragmento: até 160 bytes; até nove fragmentos por resposta.
- Frames endereçados `REQ`, `RES` e `RAK`, com ID de solicitação e numeração das partes.
- Somente a tarefa LoRa do gateway usa o SX1262; pedidos e resultados atravessam filas de quatro entradas. Consultas remotas são processadas uma de cada vez.
- Requisição inicial repetida até três vezes; cada fragmento aguarda confirmação e também possui até três tentativas de envio.
- Limite de 60 segundos por solicitação, contado desde a entrada na fila. Há uma pequena janela final para repetir confirmações do último fragmento.
- Receptor guarda somente a última requisição/resposta em RAM. Uma repetição da mesma transação reutiliza os bytes já produzidos, em vez de medir outra vez.

UDP não garante entrega. Se uma consulta expirar, repita-a; uma nova consulta recebe outro ID. Fila cheia, rádio do gateway indisponível, ausência do receptor ou resposta incompleta são informados explicitamente. Tráfego e leituras de OTA podem atrasar consultas; reinicializações interrompem transações em andamento. Atualize gateway e receptores juntos: o formato antigo `DATA`/`ACK` não é compatível.

Configuração do endereço por serial

Nos receptores, monitor serial a 115200 baud, com LF/CRLF:

```
endereco RX01
endereco?
ajuda
```

Use RX02 na segunda placa. O endereço é salvo em Preferences (`lora-config`, `endereco`), também usadas pela página web. O padrão RXXX precisa ser substituído por identificações únicas antes de ligar vários receptores no mesmo canal.

Interpretação das medições

RAM descreve heap gerenciado pelo alocador, não toda a SRAM física. Flash física não equivale a espaço livre de filesystem. Temperatura da CPU não é temperatura ambiente. `null` no JSON e `N/D` no texto indicam dados não disponíveis.

O estado de rádio acompanha operações do driver, sem leituras SPI concorrentes. RSSI/SNR são do último frame recebido com sucesso pelo rádio, que pode ter sido rejeitado pelo protocolo depois. `ULTIMO_PEER` identifica o último dispositivo com confirmação de transação. Snapshots de rádio, aplicação e sensores são protegidos individualmente; não constituem uma captura atômica de toda a placa.

`counters` conta transações e frames LoRa, não comandos locais consultados por UDP. A consulta remota observa o estado do receptor durante sua execução, antes do próprio envio da resposta terminar. Os contadores são voláteis. Os contadores de envio UDP exibidos não incluem o envio da resposta que os contém.

Bateria da Heltec V3

GPIO1 / ADC1 e chave GPIO37 ativa em LOW, divisor nominal 390 kΩ / 100 kΩ (fator 4,9). Usa `analogReadMilliVolts`, estabilização sem espera bloqueante e média de oito amostras, aproximadamente a cada cinco segundos. O fator `CALIBRACAO_BATERIA` pode ser ajustado em cada placa separadamente.

Referências: manual oficial V3 e esquemático V3.1.

Confira com multímetro: `fator_novo = fator_atual × tensao_multimetro / tensao_reportada`. A leitura não determina presença da bateria, origem USB, estado do carregador ou percentual de carga. Uma resposta `battery:RX01` usa o ADC da RX01, não o do gateway.

Compilação, teste físico com as três placas, alcance e calibração ficam com o usuário.